SARIS: Accelerating Stencil Computations on Energy-Efficient RISC-V Compute Clusters with Indirect Stream Registers

Paul Scheffler
paulsc@iis.ee.ethz.ch
Integrated Systems Lab., ETH Zurich
Zurich, Switzerland

Luca Colagrande
colluca@iis.ee.ethz.ch
Integrated Systems Lab., ETH Zurich
Zurich, Switzerland

Luca Benini
lbenini@iis.ee.ethz.ch
Integrated Systems Lab., ETH Zurich
Zurich, Switzerland
University of Bologna
Bologna, Italy

ABSTRACT

Stencil codes are performance-critical in many compute-intensive applications, but suffer from significant address calculation and irregular memory access overheads. This work presents SARIS, a general and highly flexible methodology for stencil acceleration using register-mapped indirect streams. We demonstrate SARIS for various stencil codes on an eight-core RISC-V compute cluster with indirect stream registers, achieving significant speedups of 2.72x, near-ideal FPU utilizations of 81%, and energy efficiency improvements of 1.58x over an RV32G baseline on average. Scaling out to a 256-core manycore system, we estimate an average FPU utilization of 64%, an average speedup of 2.14x, and up to 15% higher fractions of peak compute than a leading GPU code generator.

CCS CONCEPTS

• **Theory of computation** → *Parallel algorithms*; • **Mathematics of computing** → *Mathematical software performance*; • **Computing methodologies** → *Parallel algorithms*.

KEYWORDS

stencil codes, streams, ISA extensions, performance optimization

ACM Reference Format:
Paul Scheffler, Luca Colagrande, and Luca Benini. 2024. SARIS: Accelerating Stencil Computations on Energy-Efficient RISC-V Compute Clusters with Indirect Stream Registers. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3658494

1 INTRODUCTION

Stencil codes are performance-critical in numerous data-parallel, compute-intensive applications in domains like physical simulation, signal processing, and machine learning. They iterate over points in multi-dimensional data grids and update them based on their neighbors in a fixed pattern called a *stencil*.

Stencil codes share a common structure, but are highly diverse in their computational profiles, ranging from simple image filters [7] to complex seismic propagation operators [5]. The number and dimensions of I/O arrays, operations per point, and stencil shape all vary significantly between codes [8], making their acceleration on modern parallel processors challenging. Existing software proposals accelerating stencils present both generic solutions, such as code generators [6, 8, 14, 19] or platform optimizations [15, 18, 20], and tuned implementations of specific codes [5, 9]. In both cases, the target platform's architectural features are heavily leveraged.

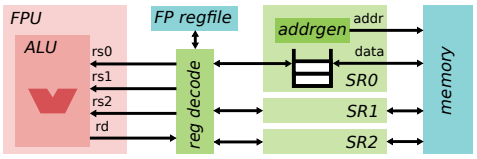


Figure 1: Integration of three floating-point SRs. When configured, the reg decode block maps accesses to registers associated with streams to SRs, which act as FIFO interfaces to memory. Addresses are produced by hardware generators.

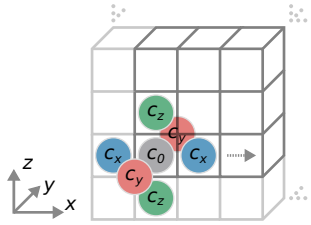
One major source of remaining inefficiencies are *memory accesses*. For instance, small stencils tend to result in low operational intensities and *memory-bound* execution [8, 12, 20]. Codes with irregular stencil shapes or many I/O arrays suffer from *irregular access patterns* inefficiently handled by tiered memory hierarchies [3, 12, 15]. Finally, both small and irregular stencils introduce significant address calculation and load-store overheads, degrading performance particularly in energy-efficient single-issue, in-order processors.

Recently, a class of hardware extensions tackling these memory inefficiencies has emerged: *stream registers* (SRs) [4, 10, 11, 13] map streams of memory accesses directly to reads or writes of architectural registers, with addresses generated by a dedicated hardware unit. A simple integration of three floating-point SRs is shown in Figure 1. SRs enable continuous streaming of useful data, maximizing bandwidth utilization and thus performance on memory-bound workloads. They also decouple memory access from computation and free the processor from handling loads, stores, and address generation, enabling near-ideal floating-point unit (FPU) utilizations in data-intensive workloads even on single-issue, in-order cores [10, 11]. Many SRs also support *indirect* streams [4, 10, 13] using a base address and index arrays to scatter or gather data, which can accelerate *arbitrarily irregular* access patterns.

In this paper, we present *SARIS*, a generic methodology for **S**tencil **A**cceleration using **R**egister-mapped **I**ndirect **S**treams. SARIS encodes the offsets of grid elements accessed in the loop body of stencil codes in index arrays; it then reuses these indices on each point update, using the point's coordinates as an indirection base. SARIS is highly flexible, applicable to *any* stencil shape, and amenable to parallelization. It leverages concurrent streams through multiple indirect SRs and additional affine SRs where beneficial. It is orthogonal to existing code optimizations including arithmetic reassociation, loop unrolling, and the use of hardware loops.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3658494



(a) Visualization of stencil shape, constant parameters, and sweep over 3D grid.

streams	*compute*
SR0 ← inp[z][y][x]	t0 = c0 × **SR0**
SR0 ← inp[z][y][x-1] **SR1** ← inp[z][y][x+1]	t1 = **SR0** + **SR1**
	t0 += cx × t1
SR0 ← inp[z][y-1][x] **SR1** ← inp[z][y+1][x]	t1 = **SR0** + **SR1**
	t0 += cy × t1
SR0 ← inp[z-1][y][x] **SR1** ← inp[z+1][y][x]	t1 = **SR0** + **SR1**
SR2 → out[z][y][x]	**SR2** += cz × t1 + t0

(b) SARIS point loop schedule showing register streams and compute operations.

Figure 2: Visualization and schedule for the symmetric 7-point star stencil code.

```
for z in 1 to N-1:
  for y in 1 to N-1:
    for x in 1 to N-1:
      out[z][y][x] =
          c0 *    inp[z][y][x]
        + cx * (  inp[z][y][x-1]
                + inp[z][y][x+1])
        + cy * (  inp[z][y-1][x]
                + inp[z][y+1][x])
        + cz * (  inp[z-1][y][x]
                + inp[z+1][y][x]);
```

(a) Baseline time iteration pseudocode.

```
x:  fld     ft0, 0(t0)      # inp[z][y][x]
    fmul.d  ft0, %[c0], ft0
    fld     ft1, -8(t0)     # inp[z][y][x-1]
    fld     ft2,  8(t0)     # inp[z][y][x+1]
    fadd.d  ft1, ft1, ft2
    fmadd.d ft0, %[cx], ft1, ft0
    fld     ft1, -YOFFS(t0) # inp[z][y-1][x]
    fld     ft2,  YOFFS(t0) # inp[z][y+1][x]
    fadd.d  ft1, ft1, ft2
    fmadd.d ft0, %[cy], ft1, ft0
    fld     ft1, 0(t1)      # inp[z-1][y][x]
    fld     ft2, 0(t2)      # inp[z+1][y][x]
    fadd.d  ft1, ft1, ft2
    fmadd.d ft0, %[cz], ft1, ft0
    fsd     ft0, 0(t3)
    addi    t0, 8
    addi    t1, 8
    addi    t2, 8
    addi    t3, 8
    bne     t0, a0, x
```

(b) Baseline point loop RISC-V assembly.

```
# Configure input offsets as indices (SR 0,1).
sr_set_idcs(SR0, {&inp[1][1][1], &inp[1][1][2],
                  &inp[1][2][1], &inp[2][1][1]});
sr_set_idcs(SR1, {&inp[1][1][0], &inp[1][0][1],
                  &inp[0][1][1] });
#Affine write stream stores outputs (SR 2).
sr_affine_write_3d(SR2, &out[1][1][1], ...);

for z in 0 to N-2:
  for y in 0 to N-2:
    for x in 0 to N-2;
      # Launch indirect input reads with
      # point offset as array base (SR 0,1)
      sr_indir_read(SR0|SR1, OFFS(x,y,z), ...);
      # Computation in order of point loop sched.
      acc =  c0 * sr_read(SR0);
      acc += cx * (sr_read(SR0) + sr_read(SR1));
      acc += cy * (sr_read(SR0) + sr_read(SR1));
      acc += cz * (sr_read(SR0) + sr_read(SR1));
      sr_write(SR2, acc);
```

(c) SARIS time iteration pseudocode.

```
x:  SRIR    SR0|SR1, t0     # SSSRs: 3 insts
    fmul.d  ft0, %[c0], SR0
    fadd.d  ft1, SR0, SR1
    fmadd.d ft0, %[cx], ft1, ft0
    fadd.d  ft1, SR0, SR1
    fmadd.d ft0, %[cy], ft1, ft0
    fadd.d  ft1, SR0, SR1
    fmadd.d SR2, %[cz], ft1, ft0
    addi    t0, 8
    bne     t0, a0, x
```

(d) SARIS point loop RISC-V assembly.

Listing 1: Time iteration pseudocode and point loop RISC-V assembly for the symmetric 7-point star stencil code with and without SARIS.

We use SARIS to optimize parallel stencil codes for the open-source, energy-efficient RISC-V Snitch compute cluster [17], which features eight RV32G cores extended with sparse stream semantic registers (SSSRs) [10] and the FREP hardware loop. We implement both platform-optimized RISC-V baseline codes and variants using SARIS to leverage the SSSR and FREP extensions, which we provide free and open-source[1]. Comparing our optimized baseline and SARIS-accelerated codes in cycle-accurate simulation, we find significant speedups of 2.72×, near-ideal FPU utilizations of 81 %, and energy efficiency improvements of 1.58× on average. We also estimate the performance benefits of SARIS on a 256-core manycore. We achieve a good mean FPU utilization of 64 %, a mean speedup of 2.14×. We obtain up to 15 % higher fractions of peak compute than a leading GPU code generator even when considering the bandwidth and latency nonidealities of a complex memory system. Our contributions are:

- We present SARIS, a generic and highly flexible approach to accelerating stencil codes with register-mapped indirect streams orthogonal to existing code optimizations.
- We implement optimized RISC-V baseline and SARIS-accelerated parallel implementations of various stencil codes on the Snitch cluster with SSSRs and FREP extensions.
- We evaluate our SARIS-accelerated codes in cycle-accurate simulation, achieving significant speedups of 2.72× over baseline codes, near-ideal FPU utilizations of 81 %, and energy efficiency improvements of 1.58× on average.
- We estimate the performance benefits of SARIS on a 256-core manycore with a bandwidth-limiting HBM2E memory stack, finding an average FPU utilization of 64 %, an average speedup of 2.14×, and up to 15 % higher fractions of peak compute than a leading GPU code generator.

[1]https://github.com/pulp-platform/snitch_cluster/tree/main/sw/saris

2 IMPLEMENTATION

We first describe our SARIS method (Section 2.1) and how it can be combined with existing code optimizations (Section 2.2). We then present our implementation of optimized baseline and SARIS-accelerated stencil codes on the Snitch cluster (Section 2.3).

2.1 SARIS Method

We describe SARIS using the example of a symmetric 7-point star stencil iterating over a single data array as shown in Figure 2a. We assume double-precision floating-point data, alternating buffers, and a grid halo initialized to sensible boundary conditions.

Listing 1a shows the baseline pseudocode for one time iteration, where `inp` represents the current and `out` the next iteration buffer. Compiling the innermost point loop for the RV32G architecture[2] without further optimizations yields the assembly code in Listing 1b. Out of 20 loop instructions, only 7 (35 %) do useful compute, while 12 (60 %) are dedicated to memory accesses and address calculation. As a result, FPU utilization is limited to 35 % of its peak on energy-efficient single-issue in-order cores[3], even when ignoring stalls due to data dependencies and irregular memory access patterns. While unrolling and register-tiling data along the x axis could eliminate up to two loads per output point, this would still yield at most 39 % FPU utilization. As we will show in Section 3.1, this limited compute efficiency manifests across all considered stencil codes.

[2]We assume here our grid tile is small enough for y neighbors to be addressed using 12 b immediate offsets, but too big to do this for z neighbors, i.e. $16 \leq N \leq 128$.
[3]While multi-issue out-of-order cores could further increase FPU utilization through instruction level parallelism, the associated energy overheads would be significant.

The SARIS method accelerates stencil computation by minimizing the above memory access overheads through the use of SRs. It involves the following steps on the point loop:

(1) Map all *grid data loads* to indirect stream reads.
(2) *Partition* these reads among available indirect SRs, maximizing their concurrent use and balancing their utilization.
(3) Map *grid data stores* or loads of *constant stencil coefficients* that cannot be kept in the register file to remaining SRs.
(4) Determine a *point loop schedule* specifying in which order the computations in the point loop access streams; this will determine the index arrays for each indirect SR.

The method may be iterated on for best performance, especially when combined with orthogonal optimizations like those described in Section 2.2. We demonstrate its steps on our 7-point-stencil code example; as on our evaluation platform, we assume the two indirect SRs (`SR0` and `SR1`) and one affine SR (`SR2`) to be available:

(1) We map all seven grid data loads to indirect stream reads.
(2) For each axis, we map the two opposing grid point loads to `SR0` and `SR1`, respectively, so they can concurrently be read by an addition operation. The remaining center point load is mapped to `SR0`, which already results in minimal utilization imbalance between `SR0` and `SR1`.
(3) Since all four stencil coefficients (c_0, c_x, c_y, c_z) can be kept in the register file, we map our single grid data store to `SR2` using an affine 3D pattern along the grid.
(4) We use the same computation order as our baseline. The resulting point loop schedule shown in Figure 2b lists each compute operation and its stream accesses in order.

The SARIS-accelerated pseudocode using our new point loop schedule is shown in Listing 1c. We first configure the static index arrays for `SR0` and `SR1`; for simplicity, we keep all indices positive by defining offsets around the iteration origin $(1, 1, 1)$ and changing the axis iteration ranges to $0 \,..\, N-2$. We then configure `SR2` with the 3D iteration needed to write our output data. Inside the point loop, we launch `SR0` and `SR1` with the grid point's address offset as the base; the indices remain the same for each grid point iteration. Finally, we perform our computation, reading and writing SRs in the order defined by our point loop schedule.

Listing 1d shows the resulting RISC-V assembly for the SARIS-accelerated point loop without further optimizations. Except for launch of `SR0` and `SR1` (3 instructions on Snitch with SSSRs), incrementing the grid point, and the loop branch, *all* loop instructions now perform useful compute, almost doubling the ratio of useful compute instructions from 35 % to 58 %. Unlike for the baseline, however, the non-compute overhead in the point loop is *static* and can significantly be reduced through complementary optimizations, enabling our near-ideal mean FPU utilizations of 81 % in Section 3.1.

Going beyond our example, SARIS can accelerate *any* stencil code as it makes very few assumptions. It supports any sequence of computations on grids of any dimensionality and size. The indirect streams enable arbitrarily shaped stencils, and since the indices include array bases, any number of I/O arrays may be streamed. Indices can even dynamically be rewritten if necessary.

In principle, SARIS could automatically be performed by the compiler whenever a fixed sequence of address offsets is repeatedly accessed with changing bases. This generalized formulation could also enable its application beyond stencil codes. In this work, we focus on the manual application of the SARIS methodology to accelerate stencil codes as it is routinely done when optimizing low-level library code; compiler inference is left as future work.

Code	Dims.	Rad.	#Loads	#Coeffs.	#FLOPs
`jacobi_2d` [7]	2D	1	5	1	5
`j2d5pt` [6]	2D	1	5	6	10
`box2d1r` [6]	2D	1	9	9	17
`j2d9pt` [6]	2D	2	9	10	18
`j2d9pt_gol` [6]	2D	1	9	10	18
`star2d3r` [6]	2D	3	13	13	25
`star3d2r` [6]	3D	2	13	13	25
`ac_iso_cd` [5]	3D	4	26	13	38
`box3d1r` [6]	3D	1	27	27	53
`j3d27pt` [6]	3D	1	27	28	54

Table 1: Implemented stencil codes sorted by FLOPs per grid point; grid loads and coefficients are also per grid point.

2.2 Complementary Optimizations

SARIS is highly flexible; to further increase FPU utilization, it can be combined with existing code optimizations:

Unrolling: by unrolling the point loop and processing blocks of points at once, the impact of SARIS' static overheads can significantly be reduced. To this end, SR streams can be extended to access grid points for multiple iterations. This also enables multi-dimensional unrolls, which can reduce parallelization imbalance.

Reordering and reassociation: by reordering indirect stream indices, point loop computations can freely be reordered and reassociated. This can improve performance by avoiding memory and dependency stalls, especially when combined with unrolling.

Hardware loops: SARIS is orthogonal to hardware loops, which reduce point iteration overheads. If they provide a separate instruction buffer, they also reduce instruction cache pressure and misses.

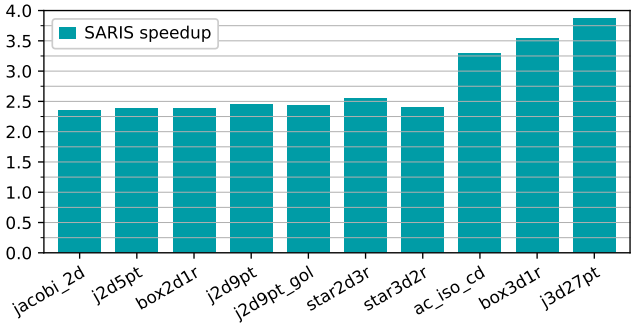
Parallelization: SARIS is compatible with parallelization at the grid point level. As every point iteration launches independent indirect streams, (unrolled blocks of) points can freely be distributed among multiple cores, for example in an interleaved fashion. Additionally, grids can be subdivided into tiles as usual.

2.3 Stencils on a Snitch Cluster

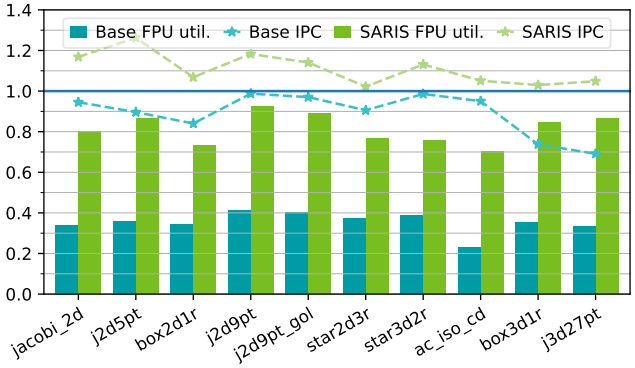
We evaluate SARIS on the open-source, energy-efficient RISC-V Snitch cluster [17], which provides eight single-issue, in-order RV32G cores. Each core consists of a minimal integer processor offloading instructions to a double-precision FPU, whose utilization can be maximized through two cooperating instruction set architecture (ISA) extensions. The *SSSR streamer* [10] provides two indirection-capable SRs (`ft0` and `ft1`) and another affine SR (`ft2`). The *FREP hardware loop* [17] provides a short repetition buffer for offloaded FPU instructions, allowing the integer processor and FPU to concurrently execute instructions in a *pseudo-dual-issue* fashion.

The Snitch cluster also provides 128 KiB of tightly coupled data memory (TCDM) across 32 banks; this memory is explicitly managed and enables high-bandwidth, low-latency accesses at 64 b granularity from all eight cores simultaneously. A 512 b programmable direct memory access (DMA) engine [1] enables high-bandwidth bulk data transfers between TCDM and main memory.

Table 1 lists the stencil codes we implement on the Snitch cluster and their performance-relevant characteristics, sorted by the number of floating-point operations (FLOPs) per grid point. All



(a) Execution speedup of SARIS over BASE code variants.



(b) FPU utilization and per-core IPC for both variants.

Figure 3: Performance comparison of BASE and SARIS stencil code variants, with codes sorted by FLOPs per grid point.

codes use double-precision data. For each, we implement one time iteration on a 64^2 (2D) or 16^3 (3D) grid tile including halos. We double-buffer the grid tiles in TCDM and program the DMA engine to concurrently transfer them from and to main memory in 2D or 3D transfers as is required for our scaleout in Section 3.3. Like our target platform, our codes are available open-source.

Each code is implemented in two parallelized variants, which are both compiled using a Snitch-optimized LLVM 15 toolchain. The *optimized baseline* (BASE) variants target the RV32G architecture without extensions; they are compiled using the `-Ofast` optimization level and a custom reassociation pass to maximize FPU utilization. The *SARIS-accelerated* (SARIS) variants target the RV32G architecture with the SSSR and FREP extensions; they also use `-Ofast`, but have manually scheduled point loops using SARIS and use FREP where possible. Both BASE and SARIS variants parallelize their point loops among the eight cluster cores using four-fold x-axis and two-fold y-axis iteration interleaving, and further unroll their point loops up to four-fold *iff* beneficial to performance.

3 EVALUATION

We first discuss the performance (Section 3.1) and energy efficiency benefits (Section 3.2) of SARIS on a single eight-core cluster. We then estimate the performance benefits of SARIS in a 256-core manycore scaleout (Section 3.3).

3.1 Performance

We execute our stencil codes in cycle-accurate simulations of the Snitch cluster's register transfer level (RTL) description. We then extract exact runtimes and utilization metrics from simulation traces.

Figure 3a shows the speedups of SARIS over BASE code variants. SARIS achieves a significant geomean speedup of 2.72×, with a clear increasing trend as FLOPs per grid point increase; the codes with the fewest (`jacobi_2d`) and most (`j3d27pt`) FLOPs per grid point also exhibit lowest and highest speedups of 2.36× and 3.87×, respectively. In between, speedups remain approximately stable up to `star3d2r` (geomean 2.42×), then increase further.

To explain this trend, we observe that codes past this point involve many *grid loads* and *coefficients* (see Table 1), increasing the register pressure in BASE variants. This reduces the benefits of unrolling, which may exhaust architectural registers and require inefficient stack accesses. However, reducing unrolling increases dependency stalls, also negatively impacting performance. SARIS avoids this register bottleneck by streaming grid points and register-exhausting coefficients directly from TCDM, maintaining the full benefits of unrolling and further increasing speedups.

Figure 3b depicts the FPU utilization and per-core IPC for both variants. As expected from our example in Section 2.1, SARIS significantly improves the geomean FPU utilization from 35 % to a near-ideal 81 %. It also enables the effective use of Snitch's pseudo-dual-issue capabilities, increasing geomean IPC from 0.89 to 1.11. SARIS performance remains consistently high, with FPU utilization and IPC never dropping below 70 % and 1.0, respectively. Variations in FPU utilization and IPC can be attributed to multiple effects. For non-register-bound codes (up to `star3d2r`), IPC variations in both variants resemble each other as both are impacted by TCDM *access contention*; codes with many *grid loads* or large *stencil overlaps* between cores exhibit more stalls due to bank conflicts. For register-bound codes (past `star3d2r`), the BASE IPC drops down to a minimum of 0.69 due to dependency stalls, while SARIS variants avoid the register bottleneck through streaming as previously explained. Finally, as the number of *grid loads* and the *stencil radius* increase, so does the setup overhead of SARIS: more indices must be stored for fewer point iterations doing useful compute. This is why `ac_iso_cd`, having the largest radius and many point loads, exhibits the minimum SARIS FPU utilization of 70 %.

Overall, the inefficiencies preventing full FPU utilization in SARIS codes are index initialization overheads, TCDM access contention, instruction cache misses, and core runtime imbalances.

3.2 Energy and Power

We implement the Snitch cluster in GlobalFoundries' 12LP+ FinFET technology using *Fusion Compiler* and execute all considered stencil codes in post-layout gate-level simulation at the cluster's target clock speed of 1 GHz. We record the resulting switching activity and use it for cluster power estimation in *PrimeTime*, assuming typical operating conditions of 25 °C and 0.8 V core supply voltage.

Figure 4 shows the resulting power consumption for each code, as well as the energy efficiency benefits of SARIS. The power consumption for both variants is fairly stable across codes with geomeans of 227 mW and 390 mW for BASE and SARIS, respectively. As would be expected, the slight variations in power consumption among either variant resemble those in FPU utilization. While the increased FPU utilization of SARIS variants results in 1.72× higher mean power consumption, the significant speedups of SARIS result in notable energy efficiency gains for all codes, ranging from 1.27× to 2.17× with

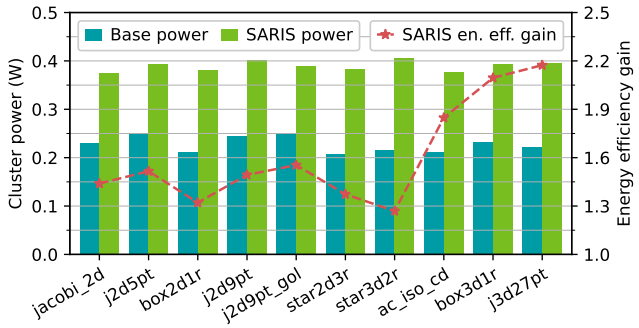


Figure 4: Cluster power consumption for BASE and SARIS variants and SARIS energy efficiency improvement over BASE.

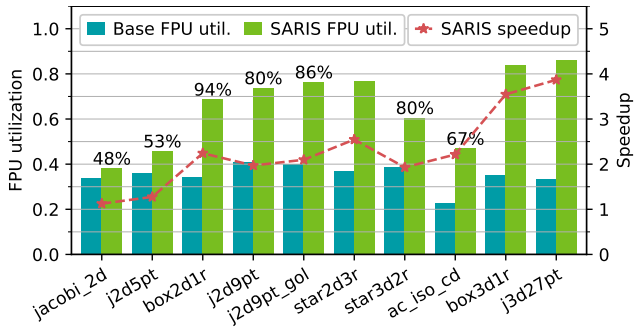


Figure 5: FPU utilization for both code variants and speedup of SARIS over BASE in the scaled-out system. The compute-to-memory time ratio is shown for memory-bound stencils.

a geomean of 1.58×. Accordingly, we observe the same increase in energy efficiency gains for register-bound codes as for speedups.

3.3 Manycore Scaleout

We estimate the performance benefits of SARIS at scale on a Snitch-based manycore system. We assume a simplified version of the Manticore [16] architecture we call *Manticore-256s*, coupling one compute chiplet to one HBM2E memory stack with eight 3.2 Gb/s/pin devices. The compute chiplet features one manager core and eight groups of four Snitch clusters each, totalling 256 cluster cores. Each group shares the bandwidth provided by one HBM2E device.

As in [6], we use grid sizes of 16384^2 for 2D and 512^3 for 3D codes. We assume the mean DMA bandwidth utilization measured in our single-cluster experiments and that available group bandwidth is shared equally among clusters. We model the imbalance in tile computation times due to scheduling variations and contention among clusters by assuming the same distribution for runtime imbalance among clusters as we observe among cores in a cluster.

Figure 5 shows the estimated FPU utilizations for the BASE and SARIS variants, as well as the estimated SARIS speedups. For memory-bound codes, we also indicate the compute-to-memory time ratio (CMTR). Despite entering the memory-bound regime for seven out of ten codes, SARIS improves the geomean FPU utilization from 35 % to 64 % and achieves a 2.14× geomean speedup over BASE variants, reaching a peak performance of 406 GFLOP/s.

For the BASE variants, FPU utilizations closely mirror the single-cluster results from Figure 3b, as all codes remain compute-bound despite bandwidth sharing among clusters. For the SARIS variants, the memory-bound codes see notable degradations in their performance. As would be expected, codes with few FLOPs per grid point exhibit a low operational intensity and thus a low CMTR, making them memory bound. As the FLOPs per grid point rises, so does the CMTR until workloads are no longer memory-bound. For the least operationally intensive 3D codes (`star3d2r` and `ac_iso_cd`), we see a regression to memory-boundedness; this is because 3D halos more strongly reduce on the ratio of input to output points in a tile, further impacting operational intensity. Moreover, `ac_iso_cd` accesses additional I/O arrays for a prior time step and a time-dependent impulse, further increasing its memory-boundedness.

Nonetheless, while SARIS yields its greatest benefits in compute-bound scenarios, it notably enhances performance even for memory-bound stencils, which, in our experiments, still show speedups as high as 2.25× over BASE variants and a geomean speedup of 1.78×.

4 RELATED WORK

While some SR proposals like *stream semantic registers* [11] and the *unlimited vector extension* [4] accelerate selected stencil benchmarks using affine streams, to the best of our knowledge, SARIS is the first generic stencil acceleration method leveraging indirect SRs to enable near-ideal FPU utilizations. Beyond SRs, numerous existing works accelerate stencil codes with dedicated hardware or through optimized CPU, GPU, and wafer-scale engine (WSE) software:

Hardware Accelerators: dedicated stencil accelerators often target FPGAs or operate near memory. *Casper* [3] accelerates stencils near a CPU's last-level cache (LLC) by leveraging its high bandwidth and data locality, incurring minimal area. *Nero* [12] is a CAPI2-interfaced FPGA+HBM solution using high-level synthesis to accelerate two compound kernels fundamental to weather prediction. *SODA* [2] automatically generates optimized dataflow architectures for stencil algorithms on FPGAs with provably optimal data reuse.

CPUs: Many CPU software approaches improve stencil performance through data layout transforms (DLTs) and vectorization. Zhang et al. [18] vectorize stencil codes for ARM's NEON extension by packing grid loads for multiple output points contiguously in memory, reaching up to 29 % of peak compute on one core of an FT-2000+ processor. Yount [15] proposes *vector folding*, a DLT forming *n*D *folds* in memory to vectorize stencils along multiple axes, using up to 30 % of a Xeon Phi 7120A's peak compute. *Bricks* [20] reorganizes large grids into fine-grain blocks contiguous in memory to exploit the full *n*D data locality of stencils and reduce TLB pressure, achieving up to 45 % of peak compute on Xeon Gold 6130.

GPUs: Numerous highly optimized code generators have been proposed for stencils on GPUs. *ARTEMIS* [8] leverages many existing and novel tiling, decomposition, fusion, and folding optimizations with parameters guided by profiling; it enables up to 36 % of peak performance on a P100 GPU. *DRStencil* [14] improves operational intensity and data reuse by fusing time steps and partitioning the resulting stencils with autotuned parameters, achieving up to 48 % of peak compute on P100. *AN5D* [6] uses a performance model to generate highly optimized CUDA code with automatic spatial and temporal blocking and shared memory (SM) double-buffering from a C source, improving performance scaling and further increasing peak compute utilization to 69 % on V100 SXM2. *EBISU* [19] leverages the much larger SM in newer GPUs by trading deep temporal blocking for lower device occupancy, achieving notable speedups over AN5D despite using only 49 % of peak compute on A100.

	Work	Platform	Prec.	% Pk.
CPU	Zhang et al. [18]	FT-2000+ (1 core)	FP64	29 %
	Yount [15]	Xeon Phi 7120A	FP32	30 %
	Bricks [20]	Xeon Gold 6130	FP32	45 %
GPU	*ARTEMIS* [8]	Tesla P100	FP64	36 %
	DRStencil [14]	Tesla P100	FP64	48 %
	AN5D [6]	Tesla V100 SXM2	FP32	69 %
	EBISU [19]	A100	FP64	49 %
WSE	Rocki et al. [9]	Cerebras WSE-1	FP16-32	28 %
	Jaquelin et al. [5]	Cerebras WSE-2	FP32	28 %
	SARIS (Ours)	Manticore-256s	FP64	**79 %**

Table 2: Overview of discussed stencil software approaches. *% Pk.* is the highest fraction of peak compute achieved.

Wafer-Scale Engines: the spatial architecture and high on-chip bandwidth of WSEs makes them attractive for stencil computation scaleouts on large grids. Rocki et al. [9] solve a large linear system arising from a 7-point stencil with mixed precision on Cerebras WSE-1, utilizing 28 % of the system's peak performance. Jacquelin et al. [5] scale out a 25-point acoustic isotropic constant-density (`ac_iso_cd` in our evaluation) seismic simulation kernel on the even larger WSE-2, mapping grid planes to tiles and streaming along the z axis; they use 28 % of their engine's peak compute.

Table 2 summarizes the discussed software works and compares the highest fraction of peak compute reported by each. We see that SARIS on our Manticore-256s scaleout reaches the highest peak performance utilization, 15 % higher than the leading GPU code generator *AN5D*. Methodologically, most of the discussed software innovations are orthogonal to SARIS; while some DLT approaches may be obviated by the fine-grain gathering abilities of indirect SRs, further optimizations improving data reuse, arithmetic intensity, locality and tiling could all benefit our Manticore-256s scaleout with SARIS, potentially enabling even higher performance.

5 CONCLUSION

We present SARIS, a flexible and generic methodology to accelerate stencil codes with indirect SRs. SARIS stores the offsets of grid point loads in static index arrays reused in each point iteration, using multiple indirect SRs for concurrent operand streaming and additional affine SRs to load coefficients or store results. It works with any stencil shape, any number of I/O arrays, and is amenable to parallelization. It can be combined with code optimizations like loop unrolling, arithmetic reassociation, and the use of hardware loops. We evaluate SARIS on the eight-core RISC-V Snitch cluster with SSSRs by implementing parallel optimized RV32G baseline and SARIS-accelerated variants of common stencil codes, which we make available open-source. SARIS achieves significant speedups of 2.72x, near-ideal FPU utilizations of 81%, and notable energy efficiency improvements of 1.58x on average. Scaling our codes to the Manticore-256s manycore, SARIS enables an FPU utilization of 64 % and a 2.14× speedup on average despite the bandwidth limitations of a complex memory system, and reaches up to 15 % higher fractions of peak compute than a leading GPU code generator.

ACKNOWLEDGMENTS

This work has been supported in part by funding from the European High-Performance Computing Joint Undertaking (JU) under Grant Agreement No 101034126 (The European Pilot) and Specific Grant Agreement No 101036168 (EPI SGA2).

REFERENCES

[1] Thomas Benz, Michael Rogenmoser, Paul Scheffler, Samuel Riedel, Alessandro Ottaviano, Andreas Kurth, Torsten Hoefler, and Luca Benini. 2024. A High-performance, Energy-efficient Modular DMA Engine Architecture. *IEEE Trans. Comput.* 73, 1 (2024), 263–277.

[2] Yuze Chi, Jason Cong, Peng Wei, and Peipei Zhou. 2018. SODA: Stencil with Optimized Dataflow Architecture. In *2018 IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*. IEEE, New York, NY, USA, 1–8.

[3] Alain Denzler, Geraldo F. Oliveira, Nastaran Hajinazar, Rahul Bera, Gagandeep Singh, Juan G'omez-Luna, and Onur Mutlu. 2021. Casper: Accelerating Stencil Computations Using Near-Cache Processing. *IEEE Access* 11 (2021), 22136–22154.

[4] Joao Mario Domingos, Nuno Neves, Nuno Roma, and Pedro Tomás. 2021. Unlimited Vector Extension with Data Streaming Support. In *2021 ACM/IEEE 48th Annu. Int. Symp. Comput. Architecture (ISCA)*. IEEE, New York, NY, USA, 209–222.

[5] Mathias Jacquelin, Mauricio Araya-Polo, and Jie Meng. 2022. Scalable Distributed High-Order Stencil Computations. In *SC '22: Proc. Int. Conf. High Perform. Comput., Netw., Storage Analysis*. IEEE Press, New York, NY, USA, Article 30, 13 pages.

[6] Kazuaki Matsumura, Hamid Reza Zohouri, Mohamed Wahib, Toshio Endo, and Satoshi Matsuoka. 2020. AN5D: Automated Stencil Framework for High-Degree Temporal Blocking on GPUs. In *Proc. 18th ACM/IEEE Int. Symp. Code Gener. Optim.* Association for Computing Machinery, New York, NY, USA, 199–211.

[7] Louis-Noël Pouchet. 2015. Polybench/C: The polyhedral benchmark suite. https://web.cse.ohio-state.edu/~pouchet.2/software/polybench/

[8] Prashant Singh Rawat, Miheer Vaidya, Aravind Sukumaran-Rajam, Atanas Rountev, Louis-Noël Pouchet, and P. Sadayappan. 2019. On Optimizing Complex Stencils on GPUs. In *2019 IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*. IEEE, New York, NY, USA, 641–652.

[9] Kamil Rocki, Dirk T. Van Essendelft, Ilya Sharapov, Robert S. Schreiber, Michael Morrison, Vladimir Kibardin, Andrey Portnoy, Jean François Dietiker, Madhava Syamlal, and Michael James. 2020. Fast Stencil-Code Computation on a Wafer-Scale Processor. In *SC20: Int. Conf. High Perf. Comput., Netw., Storage Analysis*. IEEE Press, New York, NY, USA, Article 58, 14 pages.

[10] Paul Scheffler, Florian Zaruba, Fabian Schuiki, Torsten Hoefler, and Luca Benini. 2023. Sparse Stream Semantic Registers: A Lightweight ISA Extension Accelerating General Sparse Linear Algebra. *IEEE Trans. Parallel Distrib. Syst.* 34 (2023), 3147–3161.

[11] Fabian Schuiki, Florian Zaruba, Torsten Hoefler, and Luca Benini. 2021. Stream Semantic Registers: A Lightweight RISC-V ISA Extension Achieving Full Compute Utilization in Single-Issue Cores. *IEEE Trans. Comput.* 70 (2021), 212–227.

[12] Gagandeep Singh, Dionysios Diamantopoulos, Christoph Hagleitner, Juan Gómez-Luna, Sander Stuijk, Onur Mutlu, and Henk Corporaal. 2020. NERO: A Near High-Bandwidth Memory Stencil Accelerator for Weather Prediction Modeling. In *2020 30th Int. Conf. Field-Programmable Logic Appl. (FPL)*. IEEE, New York, NY, USA, 9–17.

[13] Zhengrong Wang and Tony Nowatzki. 2019. Stream-based Memory Access Specialization for General Purpose Processors. In *2019 ACM/IEEE 46th Annu. Int. Symp. Comput. Architecture (ISCA)*. IEEE, New York, NY, USA, 736–749.

[14] Xin You, Hailong Yang, Zhonghui Jiang, Zhongzhi Luan, and Depei Qian. 2021. DRStencil: Exploiting Data Reuse within Low-order Stencil on GPU. In *2021 IEEE 23rd Int. Conf. High Perform. Comput. Commun.; 7th Int. Conf. Data Science Syst.; 19th Int. Conf. Smart City; 7th Int. Conf. Dependability in Sensor, Cloud Big Data Syst. Appl. (HPCC/DSS/SmartCity/DependSys)*. IEEE, New York, NY, USA, 63–70.

[15] Charles R. Yount. 2015. Vector Folding: Improving Stencil Performance via Multi-dimensional SIMD-vector Representation. In *2015 IEEE 17th Int. Conf. High Perform. Comput. Commun., 2015 IEEE 7th Int. Symp. Cyberspace Saf. Secur., 2015 IEEE 12th Int. Conf. Embedded Softw. Syst.* IEEE, New York, NY, USA, 865–870.

[16] Florian Zaruba, Fabian Schuiki, and Luca Benini. 2021. Manticore: A 4096-Core RISC-V Chiplet Architecture for Ultraefficient Floating-Point Computing. *IEEE Micro* 41, 2 (2021), 36–42.

[17] Florian Zaruba, Fabian Schuiki, Torsten Hoefler, and Luca Benini. 2020. Snitch: A Tiny Pseudo Dual-Issue Processor for Area and Energy Efficient Execution of Floating-Point Intensive Workloads. *IEEE Trans. Comput.* 70 (2020), 1845–1860.

[18] Kaifang Zhang, Huayou Su, Peng Zhang, and Yong Dou. 2020. Data Layout Transformation for Stencil Computations Using ARM NEON Extension. In *2020 IEEE 22nd Int. Conf. High Perform. Comput. and Commun.; IEEE 18th Int. Conf. Smart City; IEEE 6th Int. Conf. Data Science Syst. (HPCC/SmartCity/DSS)*. IEEE, New York, NY, USA, 180–188.

[19] Lingqi Zhang, Mohamed Wahib, Peng Chen, Jintao Meng, Xiao Wang, Toshio Endo, and Satoshi Matsuoka. 2023. Revisiting Temporal Blocking Stencil Optimizations. In *Proc. 37th Int. Conf. Supercomputing*. Association for Computing Machinery, New York, NY, USA, 251–263.

[20] Tuowen Zhao, Samuel Williams, Mary W. Hall, and Hans Johansen. 2018. Delivering Performance-Portable Stencil Computations on CPUs and GPUs Using Bricks. In *2018 IEEE/ACM Int. Workshop on Perform., Portability Productivity HPC (P3HPC)*. IEEE, New York, NY, USA, 59–70.